



M Ű E G Y E T E M 1 7 8 2

Budapesti Műszaki és Gazdaságtudományi Egyetem
Villamosmérnöki és Informatikai Kar
Automatizálási és Alkalmazott Informatikai Tanszék

Interaktív robotirányítás HoloLens segítségével

Szalay Gergő, Telek Marcell

Konzulens:
Dr. Hideg Attila

Budapest, 2025

Tartalomjegyzék

Bevezetés.....	2
A feladat célja.....	2
Megvalósítás.....	2
Unity scene.....	3
Alkalmazás felépítése.....	5
Osztályok.....	6
Json sorosítás.....	14
Az alkalmazás használata.....	15
Grafikus felhazsnálói felület.....	15
Megjelenítés.....	15
Irányítás.....	16
Összegzés.....	16

Bevezetés

A feladat célja

A feladat célja interaktív kiterjesztett valóság (AR) alkalmazás készítése Unity-ben Microsoft Hololens 2 kevert valóság szemüvegre, melyben a felhasználó egy Robot Operating System-et (ROS) futtató robotot irányíthat hálózaton keresztül. A robot és a szemüveg relatív helyzetét egy a roboton elhelyezett QR kód alapján szükséges meghatározni, ezután a robot LIDAR érzékelője által vett adatokat a szemüveg képes megjeleníteni a valósághoz igazítva.

Megvalósítás

A feladat megvalósításához a Unity 2022-es főverzióját használtuk, ugyanis a dokumentációk alapján ez volt a legnagyobb mértékben kompatibilis az általunk használni kívánt funkciókkal.

A Microsoft által fejlesztett Mixed Reality Toolkit (MRTK) keretrendszernek a 3-as verzióját használtuk melynek felépítése modulárisabb a korábbi kiadásokhoz képest, valamint viszonylag kevés dokumentáció állt rendelkezésünkre a verzió friss megjelenése miatt. Az alkalmazás alapját a Microsoft által kiadott hivatalos MRTK3 mintaprojekt¹ adta, ez a keretet szükséges letölteni a projekt Unity-n belüli használatához.

¹ <https://github.com/MixedRealityToolkit/MixedRealityToolkit-Unity>

Unity scene

Az alkalmazás készítésekor törekedtünk a Mixed Reality Toolkit által szolgáltatott építőelemek használatára, ahol lehetséges volt, a többi esetben saját GameObject-eket készítettünk az adott viselkedést megvalósító kód (script) csatolásával. Az alábbiakban az alkalmazásban megjelenő objektumokat mutatjuk be, azután a programkódot.

MRTK XR Rig

Ez a GameObject az MRTK Input Simulator és a EventSystem elemekkel együtt a Mixed Reality Toolkit alapvető szintér építőelemei közé tartoznak. Az EventSystem felelős a kevert valóság vezérlőkkel kapcsolatos bekövetkezett események jelzésében, ezekre a jelzésekre lehet feliratkozni. Az MRTK Input Simulator pedig az alkalmazás számítógépen, Unityben való tesztelését teszi lehetővé a bemenetek szimulációjával.

Az MRTK beépített prefabjához képest egy változtatást eszközöltünk: az MRTK XR Rig GameObject Camera Offset eleméhez hozzáadtunk egy elemet, mely tartalmazza az ARMeshManager nevű scriptet. Ez teszi lehetővé, hogy az alkalmazásunk kölcsönhatásba léphessen a valós világbeli objektumokkal (pl.: ütközések). A működéséhez szükséges egy Mesh prefab-ot is megadni, mely esetünkben egy átlátszó, Mesh collider-rel ellátott építőelem, ezt használja fel az alkalmazás a valós környezet felépítésekor alapanyagnak.

Pinch Point Detector

Ez az objektum tartalmazza a PinchPointDetector scriptnek egy példányát, aminek referenciaként a meg van adva az MRTK két ray interactorja. (Kézi gesztus érzékelők)

ROS Navigator

Ez az objektum tartalmazza a RosNavigator scriptnek egy példányát, ami felel a MRTK által átadott interakciók (PinchPointDetector) feldolgozásáért.

QR Code Manager

Ez az objektum tartalmazza a QR kód felismerését és követését, valamint a robot koordináta-rendszereinek nyilvántartását szolgáló scripteket.

A QR kódok felismerését a Mixed Reality Toolkit által szolgáltatott ARMarkerManager-t használtuk, az általa szolgáltatott adatokat pedig a saját készítésű *QR Code Manager* script veszi át és továbbítja megfelelő átalakítással a *PositionManager* osztálynak.

Websocket Manager

Tartalmazza a RosBridgeClient és WebSocketClient scriptek egy példányát, amelyek felelősek a robottal való kommunikációért és az üzenetek feldolgozásáért.

Lidar Display

Tartalmazza a LidarDisplay script egy példányát, amelynek referenciaként megadhatóak a a szkennelt pontokon megjelenítendő objektumok, annak sűrűsége, mérete és maximális száma.

Costmap Display

Tartalmazza a CostMapDisplay script egy példányát amelynek megadható az üzenetekből kinyert pontháló sűrűsége.

Path Display

Tartalmazza a PathDisplay script egy példányát és egy Line Renderer-t, amelynek a feladata a útvonaltervek megjelenítése.

Hand Menu

Az alkalmazás grafikus felhasználói felületének megvalósítása. Ezen objektum esetében kizárólag a Mixed Reality Toolkit által tartalmazott építőelemeket használtuk fel. Az elem alapbeállítása szerint az alkalmazás futásakor a felhasználó maga felé fordított, kinyújtott tenyere mellett jelenik meg, de a *GameObject* aktívra állítása esetén a számítógépes szimuláció során folyamatosan megjelenik, így egyszerűsítve a tesztelést.

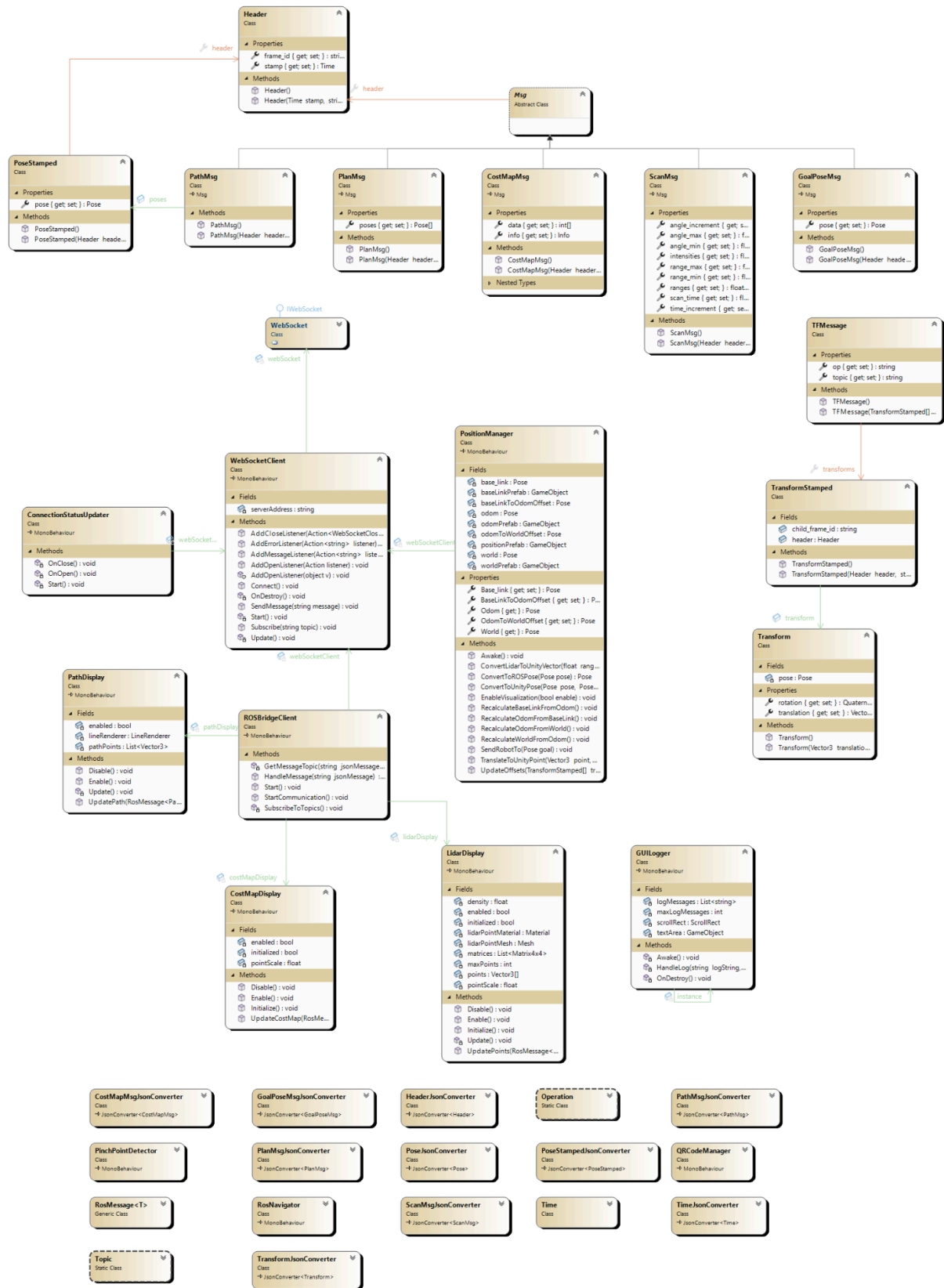
GUI Logger

A grafikus felhasználói felületen megjelenő napló-üzenetek megjelenítéséért felelős scriptet tartalmazó objektum.

Ambient Objects

Ebben a kategóriában a számítógépes tesztelés során használt elemek kerültek elhelyezésre. Részt képezte egy lap, amely a valós környezet híján a sugár célpontja volt a robot irányításához, valamint a környezeti fény. Ezeket az elemeket a Hololens-en történő futtatáshoz deaktiváltuk, ugyanis szükségtelenek, valamint sok esetben zavaróak voltak.

Alkalmazás felépítése



```

Scripts/
├── Display/                                     # Megjelenítő osztályok (pl. Lidar, Path, CostMap)
│   ├── CostMapDisplay.cs
│   ├── LidarDisplay.cs
│   └── PathDisplay.cs
├── Json Converter/                             # JSON konvertálók ROS üzenetekhez
│   └── Message/
│       ├── Primitives/                       # Alapvető üzenet típus konverterek
│           ├── HeaderJsonConverter.cs
│           ├── PoseJsonConverter.cs
│           ├── PoseStampedJsonConverter.cs
│           ├── TimeJsonConverter.cs
│           └── TransformJsonConverter.cs
│       ├── CostMapMsgJsonConverter.cs
│       ├── GoalPoseMsgJsonConverter.cs
│       ├── PathMsgJsonConverter.cs
│       ├── PlanMsgJsonConverter.cs
│       └── ScanMsgJsonConverter.cs
├── ROS/                                        # ROS-hoz kapcsolódó osztályok és adatstruktúrák
│   ├── Data/
│   │   └── Message/
│   │       ├── Primitives/                 # ROS üzenetek primitív típusai
│   │           ├── Header.cs
│   │           ├── Operation.cs
│   │           ├── PoseStamped.cs
│   │           ├── Time.cs
│   │           ├── Topic.cs
│   │           └── Transform.cs
│   │       ├── TransformStamped.cs
│   │       ├── CostMapMsg.cs
│   │       ├── GoalPoseMsg.cs
│   │       ├── Msg.cs
│   │       ├── PathMsg.cs
│   │       ├── PlanMsg.cs
│   │       ├── ScanMsg.cs
│   │       └── TFMessage.cs
│   ├── RosMessage.cs                         # Alap ROS üzenet wrapper
│   └── Network/                             # Hálózati kommunikáció ROS-szal
│       ├── ConnectionStatusUpdater.cs
│       ├── RosBridgeClient.cs
│       └── WebSocketClient.cs
├── PositionManager.cs                        # Pozíciótranszformációk Unity-hoz
├── QRCodeManager.cs                         # QR-kód kezelő logika
├── RosNavigator.cs                           # Navigáció vezérlő
├── GUILogger.cs                             # Konzol log megjelenítő UI-hoz
└── PinchPointDetector.cs                    # Egyéni logika a környezeti elemzéshez

```

Osztályok

Msg

Ez az absztrakt Msg osztály az összes ROS üzenettípus alaposztálya. Tartalmaz egy Header típusú header tulajdonságot, amely az üzenetek fejlécinformációit tárolja.

Header

A Header osztály a ROS üzenetek fejlécét valósítja meg. Két mezőt tartalmaz: az időbélyeget (stamp) és az üzenet azonosítóját (frame_id).

Operation

Az Operation osztály a ROS üzenetek művelettípusait definiálja. Két állandó értéket tartalmaz: publish és subscribe, amelyek az üzenetek küldését és fogadását jelölik.

Time

A Time osztály a ROS üzenetekben használt időformátumot reprezentálja. Másodperc (sec) és nanoszekundum (nanosec) pontosságú időadatot tárol.

Topic

A Topic osztály a ROS üzenetekben használt témanevet definiálja konstans stringként. Az egyes témák különböző típusú adatfolyamokat képviselnek:

- scan, scansim: LIDAR szenzor által gyűjtött lézerszken adatok.
- costmap: A globális költségtérkép adatai, amelyek az akadályokat és mozgási költségeket reprezentálják.
- goalpose: A robot célpozícióját tartalmazó üzenet.
- plan, plansmoothed, globalpath: A tervezett útvonal a célhoz.
tf: Helymeghatározást segítő adatok.

Transform

A Transform osztály egy 3D transzformációt reprezentál, amely tartalmaz egy eltolást (translation) és egy elforgatást (rotation). A belsőleg használt Pose objektumon keresztül kezeli ezeket az értékeket.

TransformStamped

A TransformStamped osztály egy időbélyeggel ellátott 3D transzformációt reprezentál a ROS rendszerben. Tartalmaz egy fejlécet (header), amely idő- és keretinformációkat tárol, a hozzá referenciaként megadott pontnak a id-jét (child_frame_id), valamint ennek az eltolását (transform).

PoseStamped

A PoseStamped osztály egy időbélyeggel ellátott pozíciót és orientációt (pose) reprezentál ROS üzenetként. Tartalmaz egy fejléctet (header), amely idő- és keretinformációkat tárol, valamint magát a pozíció-orientáció adatot (pose).

CostMapMsg

A CostMapMsg üzenettípus egy költségtérképről tartalmaz információkat, mint a térkép betöltési idejét, felbontását, méreteit (szélesség és magasság), valamint a térkép origóját (origin). Az Info belső osztály tárolja ezeket a metaadatokat. Az üzenet tartalmazza továbbá magát a költségtérkép adatokat (data), amely egy egész számokból álló tömb.

GoalPoseMsg

A GoalPoseMsg üzenettípus egy 3D-s pozíciót és orientációt (pose) tartalmaz, amely a robot célhelyzetét jelöli. Az üzenet az alapértelmezett fejléctet (header) örökli az Msg osztályból.

PlanMsg

A PlanMsg üzenettípus egy pozíció-orientáció (Pose) tömböt tartalmaz, amely egy robot követendő útvonalát (path) reprezentálja. Az üzenet az alapértelmezett fejléctet (header) is tartalmazza.

PathMsg

A PathMsg üzenettípus egy időbélyeggel ellátott pozíció-orientáció (PoseStamped) tömböt tartalmaz, amely egy robot követendő útvonalát reprezentálja. Az üzenet tartalmazza az alapértelmezett fejléctet (header) is.

ScanMsg

A ScanMsg üzenettípus ROS-ban a LIDAR szenzor lézerszken adatait reprezentálja. Tartalmazza a szkennelési szögtartományt (angle_min, angle_max), a szög- és időlépés értékeket (angle_increment, time_increment), a szkennelés idejét (scan_time), valamint a mérési tartományt (range_min, range_max). Az adatokat a távolságok (ranges) és intenzitások (intensities) tömbjei tárolják.

RosMessage

A RosMessage<T> egy generikus osztály, amely egy ROS üzenetet reprezentál. Tartalmazza a művelet típusát (op), a témát (topic), valamint magát az üzenetet (msg), amely egy Msg típusú objektum.

TFMessage

A /tf topic-kal címkézett üzeneteket modellező osztály. Ezen üzenetek kulcsfontosságúak a navigáció és helymeghatározás tekintetében, ugyanis ez alapján lehet a különböző koordinátarendszerek egymáshoz képesti eltérését feltérképezni.

Ennek az üzenet típusnak más az alapvető felépítése, mint a projektben fellelhető többinek, ugyanis ebben a topic és operation mezők után nem egy msg következik, hanem egy TransformStamped tömb. E lényeges különbség miatt volt szükséges az üzenetet egy teljesen különálló osztállyal modellezni, a megfelelő feldolgozhatóság miatt.

WebSocketClient

A websocketen keresztül történő kommunikációért felelős osztály. A kommunikáció megvalósításához a NativeWebSocket könyvtárat használtuk a könnyű használhatósága miatt. Publikus interfészt szolgáltat más osztályoknak a használt websocket példány eseményeire történő feliratkozásra, üzenetek küldésére és a csatlakozási folyamat elindítására.

A csatlakozást elindító Connect függvényben egy egyszerű állapotgépet valósítottunk meg, amely többszöri meghívás esetén is biztosítja, hogy a folyamat csak egyszer induljon el.

RosBridgeClient

A websocketen keresztül történő kommunikáció ROS által feldolgozható formátumra konvertálásért és a beérkező üzenetek feldolgozási folyamatának elindításáért felelős osztály.

Ezen osztályon keresztül adható ki a csatlakozási parancs a WebSocketClient osztálynak. A csatlakozást követően feliratkozási üzeneteket küld a websocketen keresztül a ROS rendszernek, majd feliratkozik a üzenetek érkezése eseményre.

Az üzenetek feldolgozása két lépésben történik: először a deszorítás elvégzése nélkül a csupán Json címkék alapján megállapítjuk az üzenet topicját, majd ezután a topic alapján megállapított RosMessage vagy TFMessage osztály megfelelő példányába beolvassuk a JSON formátumban kódolt adatokat és átadjuk az üzenet tartalmában érdekelt osztályoknak.

ConnectionStatusUpdater

Ez az osztály felelős az alkalmazáson belüli grafikus felhasználói felületen megjeleníteni a websocket kapcsolat állapotát. Feliratkozik a kapcsolódási és kapcsolat lezárása eseményekre és ezek bekövetkeztekor megváltoztatja a felhasználói felületen megjelenő szöveget.

QRCodeManager

A roboton lévő QR kód beolvasására és követésére a Mixed Reality Toolkit beépített megoldását használjuk az ARMarkerManager scripten keresztül.

Ebben az osztályban kezeljük a ARMarkerManager által felismert QR kódokat és frissítjük a marker helyzete alapján a robot aktuális helyzetét a headsethez képest. A kapcsolódó Unity objektum létrejöttkor feliratkozunk a markerek változása eseményre. Az esemény átadott értékeiben szerepel három tömb (added, updated, deleted), amelyek az újonnan felismert, a már korábban látott és most újra felismert és a törölt QR kódok tömbjeit tárolják.

A frissített QR kódok tömbjéből kiválasztjuk a robothoz tartozót (ami az "R0" szöveget tárolja) és ennek helyzetét átadjuk a PositionManager osztálynak. A helyzet rotációját -90° -kal elforgatjuk átadás előtt, ugyanis a QR kód a robot előre irányához képest jobbra merőleges helyzetben van. Az átadáskor csak az y (függőleges) tengely mentén lévő elfordulást vesszük figyelembe, ugyanis a többi a robot és az alkalmazás szempontjából érdektelen és a robot síkbeli navigációjából kimutató értékeket eredményezhet.

PositionManager

A robot pozícióját, a különböző koordinátarendszerek viszonyait és a ROS és Unity koordináta-rendszerei közötti átváltásért felelő osztály. Tárolja a ROS link-base, odom és world koordináta-rendszereinek középpontját, valamint az ezek között lévő offsetek értékét.

A link-base koordináta-rendszer középpontja maga a robot középpontja minden esetben. Ennek értékét frissíti a QRCodeManager a QR kód helyzete alapján. Ennek beállításakor automatikusan lefut az odom és world koordinátarendszerek helyzetének újraszámolása, ugyanis a Hololens szemszögéből ez az egyetlen biztos pont, aminek a helyzetét biztosan meg tudja állapítani.

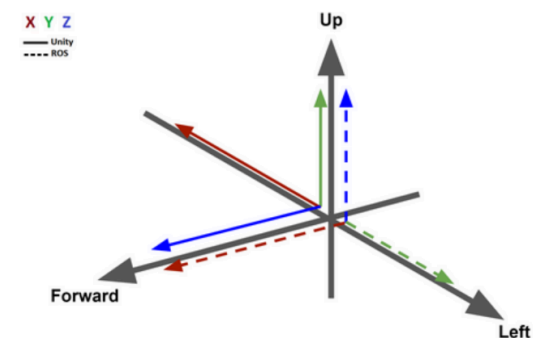
Az odom a robot odometriájának, a kerekek elfordulásának elfordulásának alapján számolt világkoordináta rendszer középpont. Az érzékelők pontatlansága miatt ez gyakran eltér a valóságtól, ezért a ROS a LIDAR érzékelő adataiból felépített térkép alapján korrigálja, ezért gyakran elmozdul.

A world a LIDAR térkép alapján meghatározott térkép szerinti világkoordináta-rendszer középpontja. Ennek elmozdulása ritkább esemény, ezért a ROS szempontjából ez tekinthető inkább fix középpontnak.

A rendszerek közötti offset a /tf topic-kal rendelkező üzenetek alapján állapítható meg. Az ebben található TransformStamped tömb ezeket az offseteket tartalmazza. Az üzeneten belül a child_frame_id határozza meg a viszonyítási pontot, a header-ben található frame_id pedig az ehhez viszonyított pont nevét, végül pedig az üzenetben tárolt transform határozza meg a helyzetet. Ezen üzenetek beérkezésekor a world koordináta alapján

újrászámolásra kerül az odom és a base_link, hogy a robot mozgása közben a QR kód folyamatos figyelése nélkül is követve legyen a robot helyzete.

A ROS jobbsodrású rendszeréből, a Unity balsodrású koordináta-rendszerébe² történő átváltás bármelyik középpont alapján lehetséges, ugyanis a különböző beérkező üzenetek különböző viszonyítási pontokkal rendelkeznek, melyet az üzenet fejlécében lévő frame_id attribútum mutat meg. A Unity rendszeréből ROS koordináta-rendszerbe történő átváltás viszont csak base_link viszonyítási ponttal került implementálásra, ugyanis jelen környezetben csak célpozíciók küldéséhez szükséges az ilyen irányú átváltás, ami base_link középpontú rendszert használ.



Koordináta konverzió vizualizálva

A beérkező LIDAR pontok koordinátáinak kiszámítása is itt törték, ebben az esetben közvetlenül a Unity koordináta-rendszerébe történik az átváltás, ezután már csak a pozíció eltolása szükséges.

Ezen osztályban megvalósításra került a koordináta-rendszerek középpontjainak vizuális megjelenítése is.

RosNavigator

A *RosNavigator* osztály a Unity környezetben a Mixed Reality Toolkit (MRTK) segítségével valós idejű navigációs vezérlést valósít meg ROS rendszerek számára. Az osztály WebSocket klienssel kommunikál a ROS-szal, és egy *PinchPointDetector* segítségével érzékeli a felhasználó ujjmozdulatait (pinch gesture).

Az Update() metódus folyamatosan figyeli, hogy a felhasználó hol hajt végre pinch-et, és amikor a pinch felengedésre kerül, a legutóbbi pinch helyét (lastPinchLocation) használja a navigációs célpont meghatározására.

A HandleNavigation metódus a pinch helyéből kinyeri a 3D pozíciót, és kiszámítja a megfelelő orientációt úgy, hogy az csak a vízszintes síkban legyen irányított (az Y komponensét nullázza). Ezt követően a pozíciót és orientációt átalakítja ROS kompatibilis formátumba (Pose), majd JSON formátumban egy GoalPoseMsg üzenetet készít, amelyet a WebSocket kliensen keresztül elküld a ROS rendszernek a /goal_pose témán.

²

https://github.com/siemens/ros-sharp/wiki/Dev_ROSUnityCoordinateSystemConversion

PinchPointDetector

A *PinchPointDetector* osztály a Mixed Reality Toolkit (MRTK) *HandsAggregatorSubsystem* rendszerét használja az ujjösszezáró (pinch) gesztusok észlelésére. Két MRTKRayInteractor objektumot kezel, amelyek a bal és jobb kéz mutató sugarait képviselik.

Az osztály fő feladata, hogy felismerje, mikor hajt végre a felhasználó pinch gesztust, és meghatározza annak pontos helyét (pinch pontját). Ezt a *DetectPinchLocation* metódus valósítja meg, amely a bal és jobb kéz ujjösszezárási állapotát ellenőrzi, és visszaadja a megfelelő ray interactort, ha pinch érzékelhető.

A *DetectPinch* metódus egyszerűen igaz/hamis értékkel jelzi, hogy jelenleg bármelyik kézen pinch gesztus történik-e. A *DetectPinchRelease* a pinch mozdulat felengedését érzékeli úgy, hogy figyeli a pinch állapotának változását. Amint a pinch felengedésre kerül, igaz értékkel tér vissza, ezzel jelezve az eseményt.

A *CalculatePreviewPointerTransform* metódus a pinch helyének vizuális megjelenítéséért felel. A pinch pont pozícióját és irányát veszi, és ennek megfelelően mozgatja és forgatja a *previewPointer* objektumot, amely a felhasználónak valós idejű visszajelzést ad a célponttól.

Az osztály egyértelmű és megbízható gesztusérzékelést tesz lehetővé, amely alapja lehet komplexebb interakcióknak vagy navigációs parancsoknak Mixed Reality környezetben.

GUILogger

A könnyebb hibakeresés érdekében a grafikus felhasználói felületen megtalálható egy opcionálisan megjeleníthető napló-üzeneteket (log) kiíró ablak is, ennek megvalósítása ebben az osztályban található.

Az alkalmazás betöltésekor feliratkozik a log üzenetek érkezésére és hozzáadja azokat a grafikus ablak szövegéhez. A könnyebb olvashatóság érdekében az üzenetek 128 karakter hosszúságra vannak korlátozva, előlött a “...” karaktersorozattal kerül helyettesítésre a fennmaradó szövegrészlet. Az ablakméret korlátozásai miatt az utolsó 50 üzenet kerül csak megjelenítésre egyszerre.

CostMapDisplay

A *CostMapDisplay* osztály a ROS (Robot Operating System) által szolgáltatott költségtérképes adatokat jeleníti meg Unity környezetben. A költségtérkép (CostMap) egy olyan mátrix, amely egy adott terület celláinak akadálymentességét vagy költségét mutatja meg, és így használható robot navigációhoz.

Funkciók és működés:

- Engedélyezés és inicializálás:
Az osztály egy *Enable()* metódussal rendelkezik, amely elindítja az inicializálást, ha az még nem történt meg, majd aktiválja a megjelenítést. A *Disable()* metódus kikapcsolja a megjelenítést. Az *Initialize()* metódus az egyszeri inicializálási lépéseket tartalmazza.
- Költségtérkép frissítése:
Az *UpdateCostMap* metódus fogad egy *RosMessage<CostMapMsg>* üzenetet. A metódus végigiterál az üzenetben lévő mátrix minden egyes pontján, kiszámolja a pont 3D-s Unity koordinátáját az eredeti térkép alapján, majd a pont költségétől függően különböző színekkel jeleníti meg azt.
- Vizualizáció:
A pontokat Unity *Debug.DrawLine* segítségével jeleníti meg kis függőleges vonalként a 3D térben. A vonalak színe a költségérték alapján változik.
- Paraméterek:
A *pointScale* változó a megjelenített pontok méretét szabályozhatja (jelenleg nincs kihasználva, de kiegészítésként használható).

LidarDisplay

Az *LidarDisplay* osztály egy Unity komponenshez tartozó script, amely a Lidar szenzor által szolgáltatott pontfelhő adatokat jeleníti meg hatékonyan, GPU instancing technikával. Ez a megközelítés lehetővé teszi, hogy nagyszámú pontot rendereljünk egyszerre, jelentős teljesítményvesztés nélkül, így ideális valós idejű megjelenítéshez.

Főbb funkciók és működés

- Pontok kezelése: A *UpdatePoints* metódus egy ROS *ScanMsg* üzenetet vár, amely a Lidar szenzor által mért távolságokat és szögeket tartalmazza. Ezekből a mérési adatokból a metódus kiszámítja a pontok 3D pozícióját Unity koordinátarendszerben.
- Sűrűségyszűrés: A pontokból eltávolítja azokat, amelyek túl közel vannak egymáshoz vagy az origóhoz, a megadott density érték alapján. Ez a szűrés csökkenti a pontok számát, hogy ne legyen túlzsúfolt a megjelenítés, és optimalizálja a teljesítményt.
- Pontok megjelenítése: Az *Update* metódus minden képkockában ellenőrzi, hogy az osztály engedélyezve van-e, majd GPU instancing segítségével rendereli a pontokat. A pontok pozícióját és méretét transzformációs mátrixokba rendezi, majd batch-ekben rajzolja ki őket.

- Testreszabhatóság: Az osztály paraméterei között megtalálható a pontok közötti minimális távolság (*density*), a pontok mérete (*pointScale*), illetve a maximum egyszerre megjeleníthető pontok száma (*maxPoints*), amely Unity korlátok miatt maximum 1023 lehet egy batch-ben.

PathDisplay

A *PathDisplay* osztály célja egy tervezett útvonal vizualizálása a Unity játékmotorban, egy *LineRenderer* komponens segítségével. A megjelenítés valós idejű ROS (Robot Operating System) üzenetek alapján történik, amelyek az útvonal pontjait tartalmazzák.

Főbb funkciók és működés

- Útvonal frissítése: Az *UpdatePath()* metódus egy ROS *PathMsg* típusú üzenetet fogad, amely tartalmazza az útvonalat alkotó pontok pozícióit. A kapott koordinátákat Unity-pozícióvá alakítja a *PositionManager.TranslateToUnityPoint()* segítségével, majd eltárolja azokat.
- Renderelés: Az *Update()* metódus minden képkockában frissíti a *LineRenderer* pozícióit a belső *pathPoints* lista alapján, amennyiben az osztály engedélyezve van (GUI-n keresztül kikapcsolható).

Json sorosítás

A projektben definiált `JsonConverter` osztályok elsődleges célja az optimalizált JSON szerializáció és deszerializáció biztosítása Unity környezetben a ROS üzenetekhez. A `Newtonsoft.Json` könyvtár alapértelmezett deszerializációja bizonyos esetekben lassú lehet, különösen komplex, nagy méretű vagy gyakran érkező adatok esetén. Ezért szükség volt egy testreszabott, gyorsabb konvertáló réteg megvalósítására.

Az alapértelmezett `Newtonsoft.Json` deszerializáció jelentős overhead-et okozott, ami hátráltatta a valós idejű adatfeldolgozást. Ezek a konverterek célzottan optimalizálták a JSON olvasást és írást, hogy minimális legyen az erőforrásigény és a késleltetés. A konverterek explicit módon kezelik az egyes mezőket, így elkerülhető a felesleges adatfeldolgozás és a túlzott reflektív műveletek, amelyek az alap `Newtonsoft.Json` működését lassíthatják.

Az üzenetek Unity típusokat (például `Vector3`, `Quaternion`, `Pose`) tartalmaznak, amelyeket a standard JSON deszerializáló nem kezel hatékonyan vagy egyáltalán nem. Egyedi konverterek segítenek az ezekhez igazított, gyorsabb feldolgozásban.

JSON konverterek összefoglalva

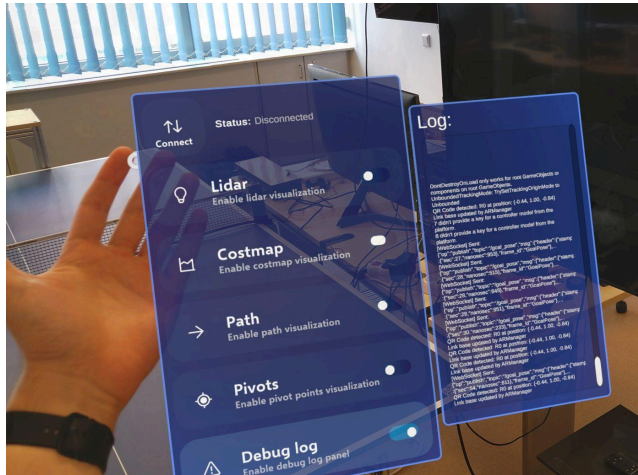
- Primitív típusok konverterei:
 - **TimeJsonConverter:** kezeli az időbélyeg mezőket (`sec`, `nanosec`), melyek ROS üzenetek alapvető részei.
 - **HeaderJsonConverter:** a *Header* üzenet szerializálását végzi, ami minden ROS üzenet fejlécét képviseli.
 - **PoseJsonConverter:** a Unity-s *Pose* típus pozícióját (`Vector3`) és orientációját (`Quaternion`) alakítja JSON formátumba.
 - **TransformJsonConverter:** a *Transform* típusból készít JSON-t, mely a pozíció és forgatás kombinációját tartalmazza.
- Összetett ROS üzenet konverterek:
 - **PoseStampedJsonConverter:** összekapcsolja a *Header* és a *Pose* típusokat egy ROS-szerű *PoseStamped* üzenetté.
 - **CostMapMsgJsonConverter:** kezeli a térképadatok (*header*, *info*, *data* tömb) JSON (de)szerializációját.
 - **GoalPoseMsgJsonConverter:** célpozíciót reprezentáló üzenet konvertere.
 - **PathMsgJsonConverter és PlanMsgJsonConverter:** útvonalakat reprezentáló üzenetekhez, melyek pozíciókat (vagy *PoseStamped* elemeket) tartalmazó listákat kezelnek.
 - **ScanMsgJsonConverter:** LIDAR érzékelő adatainak (*ranges*, *intensities* stb.) JSON feldolgozása.

Az alkalmazás használata

Grafikus felhasználói felület

A grafikus felhasználói felületet (GUI) a MRTK Hand Menu funkciójával valósítottuk meg, melynek lényege, hogy a kézfej tenyérrel felfelé fordításakor nyújtott ujjakkal, az interfész megjelenik a kézfej mellett, a kéz letétele után pedig eltűnik.

Ezen a felületen kapcsolhatjuk be a különböző vizuális megjelenítéseket, valamint a Connect gombbal elindíthatjuk a websocket csatlakozási folyamatot.



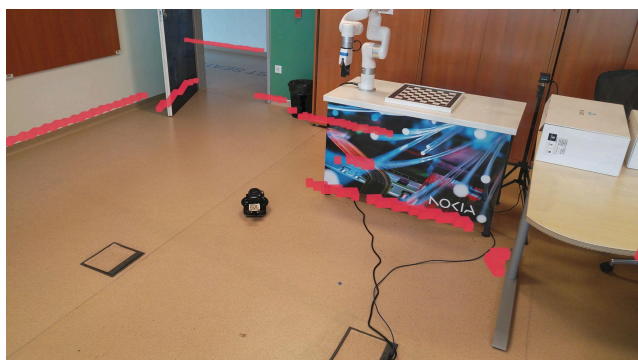
Grafikus felhasználói felület (GUI) engedélyezett napló-üzenet felülettel

Funkciók:

- Lidar: a robot LIDAR érzékelője által érzékelt pontok megjelenítése
- Costmap: a ROS által meghatározott költségtérkép megjelenítése (ez a funkció nem került teljes körűen megvalósításra)
- Path: a robot útvonaltervezése során létrejött útvonal megjelenítése
- Pivots: a ROS által használt koordináta-rendszerek középpontjainak és irányuknak megjelenítése
- Debug log: a napló-üzeneteket listázó ablak megjelenítése

Megjelenítés

Az alkalmazás képes megjeleníteni a robot LIDAR szenzora által érzékelt adatokat, ezek a pontok helyzetében megjelenő piros kockák képében jelennek meg. A optimális futás érdekében nem jelenik meg minden pont, hanem egy háló rácspontjaiban elhelyezkedő kocka akkor jelenik meg, ha az adott környezetben van megjeleníthető pont.

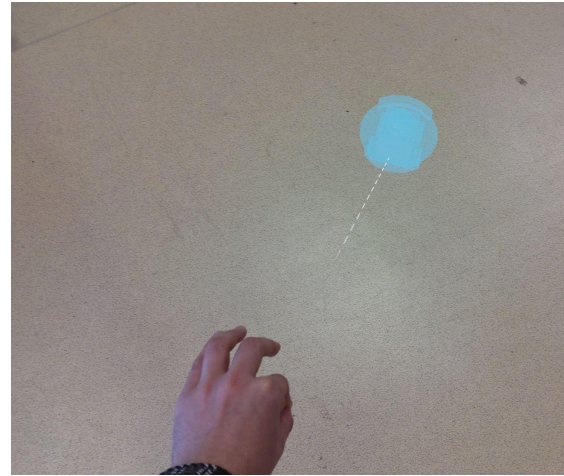


LIDAR pontok megjelenítése

Az útvonal a robot mozgása közben folyamatosan frissülve egy vonallal rajzolódik ki. A koordináta-rendszer referenciapontok bekapcsolt funkció mellett állandóan frissülve látszanak.

Irányítás

A robot irányítása a hálózati kapcsolat létrejötte és a QR kód legalább egyszeri beolvasása után lehetséges. A kézfejek végénél egyenesen egy sugár rajzolódik ki a levegőben, ezzel lehet kiválasztani a célpontot. A mutató- és hüvelykujjat összeérintve a sugár végénél megjelenik a robot áttetsző sziluettje, ekkor még mozgatható, ezzel lehet kiválasztani pontosan hova is kerüljön a célpont. Az ujjak szétválasztásakor a sziluett eltűnik, ekkor elküldésre kerül a robotnak a célpont és megkezdí a mozgást.



Összegzés

A projekt során egy interaktív kiterjesztett valóság (AR) alkalmazást fejlesztettünk Microsoft HoloLens 2-re, amely lehetővé teszi egy ROS (Robot Operating System) alapú robot távoli irányítását hálózaton keresztül. Az alkalmazás Unity motorra és a Mixed Reality Toolkit (MRTK) keretrendszerre épül, és megoldásokat nyújt a valós idejű kommunikáció, térbeli lokalizáció és vizuális megjelenítés területén.

A robot pozícióját QR kód alapú lokalizációval határoztuk meg, amely a HoloLens kamera által érzékelt roboton elhelyezett QR kód felismerésén alapul, és ennek alapján számítjuk ki a robot relatív helyzetét. A robot és a HoloLens közötti adatcsere WebSocket kapcsolaton keresztül, a ROSBridge protokoll használatával zajlik. Az alkalmazás fogadja a robot LIDAR szenzorából érkező adatokat, valamint célpozíciókat küld a navigáció támogatására.

A LIDAR pontfelhőt, a költségterképet és a robot tervezett útvonalát valós időben, a valós környezethez illesztve jelenítjük meg az AR felületen. A felhasználó kézgesztusokkal (például pinch gesture) választhat ki célpontokat, amelyeket a robot megpróbál elérni, miközben az útvonal folyamatosan frissül.

Az alkalmazás saját fejlesztésű JSON feldolgozó modulokat alkalmaz a gyorsabb üzenetfeldolgozás érdekében, ezzel minimalizálva a késleltetést és támogatva a valós idejű működést.

Fejlesztésünk során több technikai kihívással szembesültünk, így például a MRTK3 korlátozott dokumentációjával, a ROS és Unity koordinátarendszerek közötti átváltás nehézségeivel, valamint a valós idejű adatmegjelenítés teljesítményének optimalizálásával. Ezek megoldása hozzájárult a projekt sikeres megvalósításához.